Compiling Extended LAMBDA into Pure Lambda-Calculus

CS413 Midterm 1 (Assignment 6)

Mohammed Alketbi — Boston University, Fall 2026

Goal

- Translate a language with numbers, booleans, arithmetic, if, pairs, let and recursion
- into Church's pure lambda calculus: only Var, Lam, App.
- Run the output with a bounded call-by-value beta reducer and decode the answer.
- The compiler translates programs; it never evaluates them and encodes the answer.

Source and target languages

- Source (Assignment 5 AST): int n>=0, true/false, x, lam, app, not,

  + - * < <= > >= == != and or, if, let, pair, fst, snd, fix f(x).

- Target: Term ::= Var(name) | Lam(name, Term) | App(Term, Term)
- Printer: x   (lam x. body)   (fn arg)
- Semantics: left-to-right call-by-value; subtraction truncated at 0;

  if / and / or evaluate only the selected branch.

Architecture

```
target.py    — immutable AST, FV, fresh names, capture-avoiding subst, alpha-eq
church.py    — closed Church encodings (TRUE, NUM, PAIR, PRED, SUB, LT, Z ...)
compiler.py  — compile_expr: a D0ExpVisitor, never imports the evaluator
evaluator.py — ONE evaluate(e, budget) for source ASTs and target terms
observe.py   — natural / boolean / pair observers, separate budget
demo.py, examples.py, tests/ (53 unittest cases)
```

Church encodings

```
TRUE = lam t. lam f. t          FALSE = lam t. lam f. f

NUM(n) = lam s. lam z. s (... (s z))      PAIR = lam a. lam b. lam p. p a b

ADD = lam m. lam n. lam s. lam z. m s (n s z)     MUL = lam m. lam n. lam s. m (n s)

PRED = lam n. FST (n SHIFT (PAIR 0 0)),  SHIFT (a,b) = (b, b+1)  ->  PRED 0 = 0

SUB = lam m. lam n. n PRED m          (truncated: 2 - 5 = 0)

LEQ m n = ISZERO (SUB m n)   LT m n = ISZERO (SUB (SUCC m) n)   EQ = LEQ both ways
```

Translation rules

```
C[x] = x      C[lam x. e] = lam x. C[e]      C[e1 e2] = C[e1] C[e2]

C[let x = e1 in e2] = (lam x. C[e2]) C[e1]

C[if c then t else f] = (C[c] (lam u. C[t]) (lam v. C[f])) I

C[a and b] = (C[a] (lam u. C[b]) (lam v. FALSE)) I

C[a or b]  = (C[a] (lam u. TRUE) (lam v. C[b])) I

C[e1 op e2] = OP C[e1] C[e2]     C[pair a b] = PAIR C[a] C[b]

C[fix f(x). e] = Z (lam f. lam x. C[e])

negative literals, '/', other ops -> CompileError (even in dead code)
```

Why delayed branches and Z

- Call-by-value evaluates every argument before applying.
- Without thunks, if true then 1 else LOOP would evaluate LOOP -> diverge.
- Thunks (lam u. ...) are values, so passing them evaluates nothing;

  the condition picks one thunk and only that one is forced with I.
- Y f = f (Y f) unfolds forever under CBV before producing a value.
- Z eta-expands the self-application: lam v. x x v is a value,

  so the next unfolding waits until the recursive call is made.

Scope and fresh names

- Only compiler-introduced binders: thunk parameters.
- Thunk parameter = first of _k_1, _k_2, ... not occurring anywhere in its body.
- Library combinators are closed and never wrap translated code,

  so source variables named s, z, t, f, x are fine.
- let binds only in its body; fix f(f): parameter shadows function name.
- Tested: FV(C[e]) = FV(e) on open programs; input AST not mutated.

Shared evaluator

- Target: weak, left-to-right call-by-value. Values = lambdas.
- (lam x. b) v --> b[x := v] counts 1 step; renaming is free.
- Function, then argument, then contract. No reduction under lambdas.
- Budget: value at 0 steps completes; value on last step completes;

  needing a step with none left = EXHAUSTED (not proof of divergence).
- Source: same var/lam/app cases + source rules (gives expected answers).

Decoding results

```
Natural:  v STEP BASE,  STEP = lam tail. lam c. lam z. c tail,  BASE = lam c. lam z. z
  -> finite spine; host counts nodes (up to alpha-renaming).
Boolean:  v MT MF with two distinct closed markers; match the result.
Pair: evaluate, apply FST / SND, observe each part recursively.
Expected kind is required: NUM(0) and FALSE are the same term.
Statuses: ok / exhausted / malformed; separate observation budget + steps.
```

Results (exec budget 100,000 / obs budget 100,000)

```
2 + 3 * 4 = 14 (4 + 25 steps)       2 - 5 = 0       if 2 <= 3 then 7 else 9 = 7
let x = 3 in (lam y. x + y) 4 = 7    fst (pair 2 true) = 2    snd (...) = true
(lam f. f 3) (lam x. x + 1) = 4      true and false = false    false or true = true
if true then 1 else LOOP = 1         (lam x. 0) LOOP -> EXHAUSTED at 100,000
fact 4 = 24  (471 exec + 218 obs steps)     fib 5 = 5  (2,518 + 45 steps)
LOOP -> EXHAUSTED      pair (pair 2 true) 3 = ((2, true), 3)
15 / 15 demos match the expected answer and the source evaluator.
```

Testing

- 53 unittest cases:

  substitution capture/shadowing; CBV order; zero-step values; budgets

  every constructor and operator, both outcomes of every comparison,

  truth tables, pred 0, nested lets, both/coincident fix binders

  unselected divergence skipped; unused arg / let / pair part diverges

  rejection of -1, '/', unsupported unary ops inside dead code

  purity, closed combinators, compilation never calls the evaluator

Correctness and limitations

- Claim: closed, well-typed programs with natural/boolean/pair results decode to the same data after compilation, given enough budget.
- Argument: compositional rules commute with substitution of values; thunks preserve selective evaluation; Z delays recursion.
- Tests sample finitely many programs, so they are evidence, not a proof.
- Limits: no parser or type checker, unary arithmetic is slow, closed-only execution, exhaustion is not a divergence proof.